Math reference

Latin American regional preparation. C++17; zero-based indices; array ranges $[l, r)$ unless stated. Choose the model, check bounds, then type. Edition 2026-10-28.

Lookup

1 Math 1
- 1.1 Recognize the mathematical reduction 1
- 1.2 Exact arithmetic 1
- 1.3 Congruences 2
- 1.4 Prime exponents and counting values 2
- 1.5 Remainder states and decimals 2
- 1.6 Approximation as a construction 3
- 1.7 Combinatorics 3
- 1.8 Probability and expectation 3
- 1.9 Proof checks 4
- 1.10 Integer geometry 4
- 1.11 Divisors and prime exponents 4
- 1.12 GCD classes and circular motion 4
- 1.13 Prime and divisor queries: ready-to-use code . . . 5

Math (1)

1.1 Recognize the mathematical reduction

Before selecting a formula, name what the input lets you avoid. Huge decimal integers with a stated relative error may invite a bounded-prefix approximation; exact decimal text calls for integer scaling, not floating point. A rational display asks for a reduced denominator and a remainder cycle. A smallest unreachable positive sum may have a sorted-prefix invariant rather than a subset DP. A huge count of objects with a small value range often calls for frequencies and divisor classes instead of enumerating pairs. These cues are starting points: write down the exact quantity the output asks for and retain every part of the statement that changes it (ordered versus unordered, leading zeroes, strict versus inclusive error, empty choices, and modulo).

For each candidate reduction, ask four questions. What smaller object has the same answer (a prefix, a reduced denominator, a gcd class)? What invariant or identity proves that the discarded information cannot matter? What is the largest intermediate value and actual running time at the stated bound? Which helper's contract matches the inputs exactly? For instance, the decimal-shape helper handles one nonnegative fraction; it does not by itself aggregate N^2 currency pairs. Any proposed gcd-class aggregation needs its own algorithm and full-limit evidence. Likewise, factorizing a 10-digit prefix is justified by the approximation bound; that does not make arbitrary 64-bit factorization cheap.

Try the reduction on a tiny hand case and an adversarial neighbor before coding. For decimal periods, compare a terminating denominator such as 8 with one containing a coprime factor such as 14. For greedy coverage, insert a value exactly at the current boundary, then one just beyond it. If the proof needs an unstated assumption, or the tiny trace disagrees, change the model before adapting code. The worked cases in the companion chapter show this sequence: recognize, reduce, prove, trace, then map the result to a helper and its limits.

1.2 Exact arithmetic

Prelude.h

Description: Shared includes, aliases and bounds. Include before the modules.

<algorithm>, <cstdint>, <limits>, <map>, <numeric>, <optional>, <stdexcept>, <string>, <tuple>, <utility>, <vector> 17bffa, 8 lines

```
namespace finals_math {
using namespace std;
using i64 = long long;
using vi = vector<int>;
using vl = vector<i64>;
using i128 = __int128_t; // GCC/Clang

} // namespace finals_math
```

1.2.1 Floor, ceiling and bounds

Use: signed floor/ceiling division; multiples in $[L, R]$: $\max(0, \lfloor R/d \rfloor - \lceil L/d \rceil + 1)$, $d > 0$. **Pitfall:** C++ truncates toward zero. Widen *before* multiplying or subtracting: `i128(a)*b`. Helpers throw for zero denominator or overflow; 128 bits is finite.

IntegerDivision.h

Description: Checked narrowing and signed division.

"Prelude.h" cccfe3, 21 lines

```
namespace finals_math {
inline i64 narrow(i128 x) {
  if (x < numeric_limits<i64>::min() ||
      x > numeric_limits<i64>::max())
    throw overflow_error("signed 64-bit result");
  return static_cast<i64>(x);
}
// Throws for b=0 or quotient overflow.
inline i64 floor_div(i64 a, i64 b) {
  if (!b) throw invalid_argument("zero denominator");
  i128 x = a, y = b, q = x / y, r = x % y;
  if (r && ((r < 0) != (y < 0))) --q;
  return narrow(q);
}
inline i64 ceil_div(i64 a, i64 b) {
  if (!b) throw invalid_argument("zero denominator");
  i128 x = a, y = b, q = x / y, r = x % y;
  if (r && ((r < 0) == (y < 0))) ++q;
  return narrow(q);
}
} // namespace finals_math
```

1.2.2 Exact decimal input

Use: `parse_fixed(s,k)` stores $s10^k$ exactly; `digits10(x)` counts magnitude digits, including one for zero. **Requires:** $0 \le k \le 18$, integer part present, at most k fractional places, signed 64-bit result. No rounding/exponents/commas; read text, not `double`.

ExactDecimals.h

Description: Exact decimals and digit counts.

"Prelude.h", "IntegerDivision.h" 94a11e, 50 lines

```
namespace finals_math {
// Exact signed decimal; 0<=places<=18; excess digits rejected.
inline i64 parse_fixed(const string &s, int places) {
  if (places < 0 || places > 18)
    throw invalid_argument("decimal places");
  size_t j = 0;
  bool neg = false;
  if (j < s.size() && (s[j] == '+' || s[j] == '-'))
    neg = s[j++] == '-';
  i128 value = 0;
  int whole = 0, frac = 0;
  const i128 cap = i128(numeric_limits<i64>::max()) + 1;
  auto digit = [&](char c) {
    if (c < '0' || c > '9')
      throw invalid_argument("decimal digit");
    value = value * 10 + (c - '0');
    if (value > cap) throw overflow_error("decimal value");
  };
  while (j < s.size() && s[j] != '.') {
    digit(s[j++]);
    ++whole;
  }
  if (!whole) throw invalid_argument("integer part");
  if (j < s.size()) {
    ++j;
    while (j < s.size()) {
      digit(s[j++]);
      ++frac;
    }
  }
  if (frac > places)
    throw invalid_argument("too many fractional digits");
  while (frac++ < places) {
    value *= 10;
    if (value > cap) throw overflow_error("decimal value");
  }
  return narrow(neg ? -value : value);
}
// Magnitude digits: zero has one; sign excluded.
inline int digits10(i64 x) {
  i128 v = x;
  if (v < 0) v = -v;
  int d = 1;
  while (v >= 10) {
    v /= 10;
    ++d;
  }
  return d;
}
} // namespace finals_math
```

1.3 Congruences

1.3.1 Powers and inverses

Use: normalize, multiply and exponentiate modulo $m > 0$; $O(\log e)$ power, $e \geq 0$. $0^0 = 1$ before reduction. **Pitfall:** division requires $\gcd(a, m) = 1$; $a^{-1} = a^{p-2} \pmod p$ requires prime p, $p \nmid a$.

ModularArithmetic.h

Description: Modular arithmetic and binary powers.

"Prelude.h" 0b8b9c, 21 lines

```
namespace finals_math {
inline i64 norm(i64 a, i64 m) {
  if (m <= 0)
    throw invalid_argument("positive modulus required");
  a %= m;
  return a < 0 ? a + m : a;
}
inline i64 mul_mod(i64 a, i64 b, i64 m) {
  return i64(i128(norm(a, m)) * norm(b, m) % m);
}
inline i64 pow_mod(i64 a, uint64_t e, i64 m) {
  a = norm(a, m);
  i64 r = 1 % m;
  while (e) {
    if (e & 1) r = mul_mod(r, a, m);
    a = mul_mod(a, a, m);
    e >>= 1;
  }
  return r;
}
} // namespace finals_math
```

1.3.2 Bezout and shared-factor CRT

$ax + by = g = \gcd(a, b)$; $ax + by = c$ solvable iff $g \mid c$. All solutions: $(x_0 + tb/g, y_0 - ta/g)$; intersect bounds using floor/ceiling. For $a = b = 0$, handle $c = 0$ separately.

Use: merge $X = a \pmod m$, $X = b \pmod n$ even with shared factors. $g \mid (b - a)$ required; solve $(m/g)t = (b - a)/g \pmod{n/g}$, then $X = a + mt$, modulo $\operatorname{lcm}(m, n)$. **Requires:** positive moduli; nonnegative Euclid inputs. Incompatible CRT returns `nullopt`; overflowing lcm throws.

EuclidCRT.h

Description: Extended Euclid, inverses and CRT.

"Prelude.h", "IntegerDivision.h", "ModularArithmetic.h" 9f1514, 48 lines

```
namespace finals_math {
// Requires nonnegative a,b. Returns g,x,y with a*x+b*y=g.
inline tuple<i64, i128, i128> extended_gcd(i64 a, i64 b) {
  if (a < 0 || b < 0)
    throw invalid_argument("nonnegative gcd inputs");
  i128 r = a, s = b, x = 1, y = 0, u = 0, v = 1;
  while (s) {
    i128 q = r / s;
    i128 t = r - q * s;
    r = s;
    s = t;
    t = x - q * u;
    x = u;
    u = t;
    t = y - q * v;
    y = v;
    v = t;
  }
  return {i64(r), x, y};
}
inline optional<i64> inverse(i64 a, i64 m) {
  if (m <= 1)
    throw invalid_argument("inverse modulus must exceed one");
  auto [g, x, y] = extended_gcd(norm(a, m), m);
  (void)y;
  if (g != 1) return nullopt;
  return i64((x % m + m) % m);
}
struct Congruence {
  i64 residue, modulus;
};
// Merge x=a (mod m), x=b (mod n); nullopt = incompatible.
// Positive moduli. Throws if the merged lcm exceeds LLONG_MAX.
inline optional<Congruence> crt(Congruence a, Congruence b) {
  a.residue = norm(a.residue, a.modulus);
  b.residue = norm(b.residue, b.modulus);
  auto [g, x, y] = extended_gcd(a.modulus, b.modulus);
  (void)y;
  i128 delta = i128(b.residue) - a.residue;
  if (delta % g) return nullopt;
  i64 n = b.modulus / g;
  i128 t = (delta / g * x) % n;
  if (t < 0) t += n;
  i64 lcm = narrow(i128(a.modulus) * n);
  i128 r = (i128(a.residue) + i128(a.modulus) * t) % lcm;
  return Congruence{i64(r), lcm};
}
} // namespace finals_math
```

1.4 Prime exponents and counting values

1.4.1 Factorization and totient

For $n = \prod p^{e_p}$:

$$\tau(n) = \prod (e_p + 1), \quad \sigma(n) = \prod \sum_{j=0}^{e_p} p^j, \quad \phi(n) = n \prod_{p \mid n} (1 - 1/p).$$

Use: positive inputs; trial factorization $O(\sqrt{n})$, SPF preprocessing $O(V)$ time/space, factor $x \leq V$ in $O(\log x)$. **Pitfall:** repeated arbitrary 64-bit inputs need faster factorization; $n = 1$ has empty factors. Bound divisor sums/products separately.

FactorSieve.h

Description: Trial factorization, totients and sieve.

"Prelude.h" ba3fd8, 58 lines

```
namespace finals_math {
// n>0; O(sqrt(n)).
inline vector<pair<i64, int>> factor(i64 n) {
  if (n < 1) throw invalid_argument("positive factor input");
  vector<pair<i64, int>> f;
  for (i64 p = 2; p <= n / p; p += (p == 2 ? 1 : 2))
    if (n % p == 0) {
      int e = 0;
      do {
        n /= p;
        ++e;
      } while (n % p == 0);
      f.push_back({p, e});
    }
  if (n > 1) f.push_back({n, 1});
  return f;
}
inline i64 phi(i64 n) {
  i64 ans = n;
  for (auto [p, e] : factor(n)) {
    (void)e;
    ans = ans / p * (p - 1);
  }
  return ans;
}
struct Sieve {
  vi spf, primes;
  explicit Sieve(int n) {
    if (n < 0)
      throw invalid_argument("nonnegative sieve limit");
    spf.resize(size_t(n) + 1, 0);
    for (int i = 2; i <= n; ++i) {
      if (!spf[i]) {
        spf[i] = i;
        primes.push_back(i);
      }
      for (int p : primes) {
        if (p > spf[i] || p > n / i) break;
        spf[p * i] = p;
      }
    }
  }
  vector<pair<i64, int>> factors(int x) const {
    if (x < 1 || x >= int(spf.size()))
      throw invalid_argument("sieve range");
    vector<pair<i64, int>> f;
    while (x > 1) {
      int p = spf[x], e = 0;
      do {
        x /= p;
        ++e;
      } while (x > 1 && spf[x] == p);
      f.push_back({p, e});
    }
    return f;
  }
};
} // namespace finals_math
```

1.4.2 Histogram and exact gcd classes

Positive values $\leq V$: $f[x]$ = frequency, $M[d] = \sum_{k \geq 1} f[kd]$; all M cost $O(V \log V)$. For each fixed b, process divisors decreasing:

$$E_b[g] = M[g] - \sum_{\substack{h \mid b,\ g \mid h \\ h > g}} E_b[h].$$

$E_b[g]$ counts $\gcd(a, b) = g$; ordered occurrence pairs multiply by $f[b]$. **Pitfall:** zero separately; divisor subtraction adds work (e.g. $\sum_b \tau(b)^2$).

1.5 Remainder states and decimals

1.5.1 Digits as a small state

Append d: $r' = (10r + d) \bmod m$. **Use:** DP (position, r), $O(10Lm)$, only when equal states have identical legal suffixes. Add other distinguishing history to state. **Pitfall:** leading zeros; widen $10r$. Smallest reconstruction tries digits ascending with suffix feasibility, not counts alone.

1.5.2 Period and rational display

$\mathrm{ord}_m(a) \mid \phi(m)$ for $m \geq 2$, $\gcd(a, m) = 1$. **Use:** start $k = \phi(m)$; divide prime factors p while $a^{k/p} = 1 \pmod m$.
Trial-factorization cost applies.

Reduce $q = b/\gcd(a, b) = 2^u 5^v r$, $\gcd(r, 10) = 1$:

$$D(a, b) = \mathrm{digits}(\lfloor a/b \rfloor) + \max(u, v) + P(r),$$

$P(1) = 0$, otherwise $P(r) = \mathrm{ord}_r(10)$. **Why:** long division removes the $2, 5$ prefix; recurring remainders then require $10^k = 1 \pmod r$. **Requires:** $a \geq 0, b > 0$; canonical decimal, no point/overbar/trailing zeros. 2025 D aggregation: gcd classes for fractions; suffix counts at $a \geq 10b, 100b, \ldots$ for integer digits. Maximum-size aggregation unbenchmarked.

DecimalPeriod.h

Description: Multiplicative order and decimal shape.

"Prelude.h", "ExactDecimals.h", "ModularArithmetic.h", "FactorSieve.h" 4d9be3, 35 lines

```
namespace finals_math {
inline i64 order_mod(
    i64 a,
    i64 m) { // m>=2, gcd(a,m)=1; trial division cost applies.
  if (m < 2 || gcd(norm(a, m), m) != 1)
    throw invalid_argument("order requires a unit");
  i64 k = phi(m);
  for (auto [p, e] : factor(k)) {
    (void)e;
    while (k % p == 0 && pow_mod(a, k / p, m) == 1) k /= p;
  }
  return k;
}
struct DecimalShape {
  int integer_digits, nonrepeating;
  i64 period;
};
// a>=0, b>=1; period computation may be costly.
inline DecimalShape decimal_shape(i64 a, i64 b) {
  if (a < 0 || b < 1)
    throw invalid_argument("nonnegative fraction");
  i64 q = b / gcd(a, b);
  int u = 0, v = 0;
  while (q % 2 == 0) {
    q /= 2;
    ++u;
  }
  while (q % 5 == 0) {
    q /= 5;
    ++v;
  }
  return {digits10(a / b), max(u, v),
          q == 1 ? 0 : order_mod(10, q)};
}
} // namespace finals_math
```

1.6 Approximation as a construction

1.6.1 Fuzzy Factorization (2025 F)

Keep ten leading digits: $X = m10^e + r$, $0 \leq r < 10^e$, $m \geq 10^9$. Choose $Y = m10^e$: $|X - Y|/X < 1/m \leq 10^{-9}$. **Use:** factor $m < 10^{10}$; add e to exponents of $2, 5$. At most ten digits: exact factorization. $O(|X| + \sqrt{m})$. **Requires:** positive canonical decimal text. Empty map means one; merge primes and adapt output format. Different tolerance needs a new bound.

FuzzyFactorization.h

Description: Fuzzy factorization construction.

"Prelude.h", "FactorSieve.h" 41a1d5, 21 lines

```
namespace finals_math {
// Positive decimal, no leading zero; empty factors means Y=1.
inline map<i64, int> fuzzy_factors(const string &s) {
  if (s.empty() || s[0] == '0')
    throw invalid_argument("positive canonical decimal");
  for (char c : s)
    if (c < '0' || c > '9')
      throw invalid_argument("decimal digits");
  i64 m = 0;
  int keep = int(min<size_t>(10, s.size()));
  for (int i = 0; i < keep; ++i) m = m * 10 + s[i] - '0';
  int e = int(s.size()) - keep;
  map<i64, int> f;
  for (auto [p, k] : factor(m)) f[p] = k;
  if (e) {
    f[2] += e;
    f[5] += e;
  }
  return f;
}
} // namespace finals_math
```

1.7 Combinatorics

1.7.1 Sample objects and binomial coefficients

Disjoint cases add; exactly A choices then B each multiply. Ordered distinct: $n!/(n-k)!$; subsets: $\binom{n}{k}$; ordered repetition: n^k. Stars/bars, $k \geq 1$: nonempty bins $\binom{n-1}{k-1}$ $(n \geq k)$; empty allowed $\binom{n+k-1}{k-1}$. Pascal: $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$ (any modulus). **Use:** factorial tables, $O(N + \log p)$ build, $O(1)$ query. **Requires:** prime p, $N < p$; primality unchecked. No Lucas/composite support.

Combinations.h

Description: Combinations under a prime modulus.

"Prelude.h", "ModularArithmetic.h" 10ccf0, 25 lines

```
namespace finals_math {
// Requires prime p and n<p; primality is unchecked.
struct PrimeCombinations {
  i64 p;
  vl fac, invfac;
  PrimeCombinations(int n, i64 prime) : p(prime) {
    if (n < 0 || p <= 1 || n >= p)
      throw invalid_argument("need 0<=n<prime");
    fac.resize(n + 1, 1);
    invfac.resize(n + 1, 1);
    for (int i = 1; i <= n; ++i)
      fac[i] = mul_mod(fac[i - 1], i, p);
    invfac[n] = pow_mod(fac[n], p - 2, p);
    for (int i = n; i > 0; --i)
      invfac[i - 1] = mul_mod(invfac[i], i, p);
  }
  i64 choose(int n, int k) const {
    if (n < 0 || n >= int(fac.size()))
      throw invalid_argument("choose range");
    if (k < 0 || k > n) return 0;
    return mul_mod(fac[n],
                   mul_mod(invfac[k], invfac[n - k], p), p);
  }
};
} // namespace finals_math
```

1.7.2 Contributions and complements

$$\sum_{\text{subarrays}} \sum a_i = \sum_i a_i(i+1)(n-i),$$

$$\sum_{i<j}(a_i + a_j) = (n-1)\sum a_i, \quad \sum_{i<j}(a_i - a_j)^2 = n\sum a_i^2 - (\sum a_i)^2.$$

At least one good = total − zero good. Bad run length L: $L(L+1)/2$ all-bad substrings. **Pitfall:** wide products; circular boundaries/all-bad case separately.

1.7.3 Inversions and repeated copies

Use: strict inversions $i < j, a_i > a_j$; compression/Fenwick, $O(n \log n)$ time, $O(n)$ space. Equal values do not count.

$$\mathrm{inv}(S^K) = K \text{ within} + \binom{K}{2} \text{ cross}.$$

cross $= \#\{(u, v) : S_u > S_v\}$ with no within-copy index restriction (2023 I). Widen/modularize products.

Inversions.h

Description: Strict inversions by compressed Fenwick tree.

"Prelude.h", "IntegerDivision.h" 5e519b, 21 lines

```
namespace finals_math {
// Counts i<j with a[i]>a[j]; O(n log n); input unchanged.
inline i64 inversions(vl a) {
  auto vals = a;
  sort(vals.begin(), vals.end());
  vals.erase(unique(vals.begin(), vals.end()), vals.end());
  vl bit(vals.size() + 1);
  i64 answer = 0, seen = 0;
  for (i64 x : a) {
    int k = int(lower_bound(vals.begin(), vals.end(), x) -
                vals.begin()) +
            1;
    i64 le = 0;
    for (int j = k; j; j -= j & -j) le += bit[j];
    answer = narrow(i128(answer) + seen - le);
    ++seen;
    for (int j = k; j < int(bit.size()); j += j & -j) ++bit[j];
  }
  return answer;
}
} // namespace finals_math
```

1.8 Probability and expectation

$$\mathbb{E}[X] = \sum_x x \Pr(X = x), \quad \mathbb{E}[\sum I_j] = \sum \Pr(I_j = 1).$$

Linearity needs no independence; $\Pr(A \cap B) = \Pr(A)\Pr(B \mid A)$. Ordered occurrence pairs: N^2 with replacement, $N(N-1)$ without. $E = c + qE + \sum p_j E_j \Rightarrow E = (c + \sum p_j E_j)/(1-q)$, $q < 1$. **Pitfall:** modular A/B requires invertible reduced denominator; never convert rounded decimals.
$\mathrm{Var}(X) = E[X^2] - E[X]^2$.

1.9 Proof checks

Invariant: initialize, preserve, then prove sufficiency by construction. Termination: decreasing nonnegative integer. Binary search: monotone predicate. Greedy: feasibility-preserving exchange. Tiny brute force finds counterexamples. $\sum i = n(n+1)/2$; $\sum_{i=0}^{k} r^i = (r^{k+1}-1)/(r-1)$, $r \neq 1$. Modular division needs cancellation/invertibility; error uses original denominator.

1.9.1 Greedy coverage: smallest missing subset sum

Positive integers sorted ascending; initially $R = 0$ and every sum in $[0, R]$ is reachable. If $x > R+1$, answer $R+1$; otherwise extend $R \leftarrow R + x$. After all values answer $R+1$. Proof: old sums $[0, R]$ and sums using x, $[x, x+R]$, have no integer gap iff $x \leq R+1$. Masters Mexico 2022 H; $O(n \log n)$; total plus one must fit.

```
long long missing(std::vector<long long> a) {
  std::sort(a.begin(), a.end());
  long long next = 1;
  for (auto x : a) {
    assert(x > 0);
    if (x > next) break;
    next += x;
  }
  return next;
}
```

1.10 Integer geometry

$u \cdot v = u_x v_x + u_y v_y$; $u \times v = u_x v_y - u_y v_x$. orient$(A, B, C) = (B - A) \times (C - A)$: positive left, zero collinear. $2A_{\triangle} = |\,\text{orient}\,|$; compare squared distances. **Use:** exact predicates; segment membership = collinear + inclusive bounds. Simple polygon in boundary order: $2A = |\sum P_i \times P_{i+1}|$. **Requires:** helper coordinates $|x|, |y| \leq 10^{18}$; polygon sum needs its own bound. Compare orientation signs, not multiplied crosses; no universal floating EPS.

IntegerGeometry.h

Description: Exact integer geometry.

"Prelude.h" 45bdbc, 17 lines

```
namespace finals_math {
struct Point {
  i64 x, y;
};
// |coordinate|<=1e18 keeps these predicates within i128.
inline i128 orient(Point a, Point b, Point c) {
  return (i128(b.x) - a.x) * (i128(c.y) - a.y) -
         (i128(b.y) - a.y) * (i128(c.x) - a.x);
}
inline i128 dot(Point a, Point b) {
  return i128(a.x) * b.x + i128(a.y) * b.y;
}
inline i128 dist2(Point a, Point b) {
  i128 x = i128(a.x) - b.x, y = i128(a.y) - b.y;
  return x * x + y * y;
}
} // namespace finals_math
```

Prelude.h

Description: Shared includes, aliases and bounds. Include before the modules.

<algorithm>, <cstdint>, <limits>, <map>, <numeric>, <optional>, <stdexcept>, <utility>, <vector> 172723, 5 lines

```
namespace finals_math_extra {
using namespace std;
using i64 = long long;

} // namespace finals_math_extra
```

1.11 Divisors and prime exponents

Choose the method first. One moderate positive integer can use trial division, removing each full prime power in $O(\sqrt{n})$. Many queries bounded by a feasible V favor an SPF sieve; its memory is $O(V)$. Trial division is a poor choice for many large inputs or adversarial semiprimes. Neither method is unrestricted 64-bit factorization.

1.11.1 Generate divisors

positive_divisors(n) returns every positive divisor in sorted order. It scans $d \leq n/d$ and pairs d with n/d, so it does not construct divisors from a factorization. Runtime is $O(\sqrt{n})$ plus output storage. Requires $n > 0$.

Divisors.h

Description: Divisors.

"Prelude.h" d91b38, 17 lines

```
namespace finals_math_extra {
// Positive divisors in increasing order; O(sqrt(n)) scanning.
// Trial division is not for adversarial large 64-bit
// semiprimes.
inline vector<i64> positive_divisors(i64 n) {
  if (n < 1) throw invalid_argument("positive divisor input");
  vector<i64> lo, hi;
  for (i64 d = 1; d <= n / d; ++d) {
    if (n % d) continue;
    lo.push_back(d);
    if (d != n / d) hi.push_back(n / d);
  }
  reverse(hi.begin(), hi.end());
  lo.insert(lo.end(), hi.begin(), hi.end());
  return lo;
}
} // namespace finals_math_extra
```

1.11.2 Reason with exponents

If $n = \prod p^{e_p}$, the least multiplier making n a kth power is $\prod p^{(k - e_p \bmod k) \bmod k}$. For $72 = 2^3 \cdot 3^2$, this gives 2 for squares. Pass the factorization from a suitable method; the helper returns no value if the multiplier overflows signed 64-bit. For factorial divisibility, Legendre's sum $v_p(n!) = \lfloor n/p \rfloor + \lfloor n/p^2 \rfloor + \cdots$ avoids constructing $n!$. Its helper requires prime p.

PrimeExponents.h

Description: PrimeExponents.

"Prelude.h" da21fc, 32 lines

```
namespace finals_math_extra {
// Multiplier exponent vector making n a kth power.
// f has distinct PRIME bases and positive exponents;
// unchecked. k positive; nullopt means product exceeds signed
// 64-bit.
inline optional<i64>
kth_power_multiplier(const vector<pair<i64, int>> &f, int k) {
  if (k <= 0) throw invalid_argument("positive power");
  i64 ans = 1;
  for (auto [p, e] : f) {
    if (p < 2 || e <= 0)
      throw invalid_argument("prime exponent");
    int add = (k - e % k) % k;
    while (add--) {
      if (ans > numeric_limits<i64>::max() / p) return nullopt;
      ans *= p;
    }
  }
  return ans;
}
// Legendre formula; p must be prime and n>=0.
inline i64 factorial_valuation(i64 n, i64 p) {
  if (n < 0 || p < 2)
    throw invalid_argument("valuation input");
  i64 total = 0;
  while (n) {
    n /= p;
    total += n;
  }
  return total;
}
} // namespace finals_math_extra
```

1.12 GCD classes and circular motion

1.12.1 Count exact pairwise GCDs

Let M_d count input values divisible by d. Every unordered pair among those M_d values has gcd a multiple of d. Thus, descending from V, subtract counts already assigned to proper multiples: $E_d = \binom{M_d}{2} - \sum_{j \geq 2} E_{jd}$. Then E_d counts pairs with gcd exactly d, and E_1 is the coprime-pair count. This handles duplicate values by index and excludes pairing an item with itself. Requires positive values at most V; time $O(V \log V + N)$ and memory $O(V)$. For a small number of pairs or a huge value range, use direct pairwise gcd in $O(N^2 \log V)$ instead. For $N < 2$, every answer is 0.

GCDPairs.h

Description: GCDPairs.

"Prelude.h" c2803e, 31 lines

```
namespace finals_math_extra {
// Exact gcd counts for unordered distinct-index pairs.
// Values positive and <=V; N<=INT_MAX; O(V log V + N).
inline vector<i64> gcd_pair_histogram(const vector<int> &a,
                                      int V) {
  if (V < 1) throw invalid_argument("positive value bound");
  vector<i64> freq(size_t(V) + 1), exact(size_t(V) + 1);
  for (int x : a) {
    if (x < 1 || x > V)
      throw invalid_argument("gcd value range");
    ++freq[x];
  }
  for (int d = V; d >= 1; --d) {
    i64 count = 0;
    for (i64 m = d; m <= V; m += d) count += freq[m];
    exact[d] = count * (count - 1) / 2;
    for (i64 m = 2LL * d; m <= V; m += d) exact[d] -= exact[m];
  }
  return exact;
}
// Vacuously true for fewer than two entries.
```

```
// Positive inputs, affordable V; same bounds as histogram.
inline bool same_pair_gcd(const vector<int> &a, int V) {
  auto h = gcd_pair_histogram(a, V);
  i64 n = a.size();
  if (n < 2) return true;
  int g = 0;
  for (int x : a) g = gcd(g, x);
  return h[g] == n * (n - 1) / 2;
}
} // namespace finals_math_extra
```

1.12.2 Are all pairwise GCDs equal?

The supplied pair loop is correct for $N \geq 2$, but costs $O(N^2 \log V)$ and reads out of bounds when $N < 2$. Decide that boundary from the statement: with no pairs, "all pairs agree" is vacuously true, although there is no unique pairwise gcd value. A safe small-input version is:

```
// Positive values. O(n^2 log(max a)).
bool same_gcd(const vector<long long>& a) {
    if (a.size() < 2) return true;
    long long g = std::gcd(a[0], a[1]);
    for (int i = 0; i < (int)a.size(); ++i)
        for (int j = i + 1; j < (int)a.size(); ++j)
            if (std::gcd(a[i], a[j]) != g)
                return false;
    return true;
}
```

For bounded positive values, `same_pair_gcd(a,V)` uses the histogram. If every pair agrees, that common value must equal the gcd of the whole array; check whether its exact class contains all $N(N-1)/2$ pairs. Computing the whole-array gcd alone is insufficient: $[6, 10, 15]$ has global gcd 1 but pair gcds 2,3,5. On $[6, 10, 14]$ every pair gcd is 2; on $[2, 2, 4]$ it is also 2, so duplicates need not fail.

An alternative when SPF is already built: divide each number by the global gcd, factor it, and reject if a prime appears in two different numbers. The normalized numbers must be pairwise coprime. Mark each distinct prime once per number; normalized 1 has no primes and may repeat. This is $O(V + N \log V)$ with the existing linear SPF sieve, but the histogram is more useful when the problem asks for counts of several gcd values.

1.12.3 Orbits under repeated addition

On n circular positions, repeatedly adding s forms $\gcd(n, s)$ disjoint cycles, each of length $n/\gcd(n, s)$. For $(n, s) = (12, 8)$, there are four cycles of length three; from zero the visited positions are $0, 8, 4$. Reduce reachability to the residue class modulo the gcd, then check any safety condition along that cycle.

GCDOrbits.h

Description: GCDOrbits.

`"Prelude.h"` 76dcd9, 11 lines

```
namespace finals_math_extra {
// Addition by step on Z/nZ: gcd(n,step) cycles of this length.
// n>0; step may be any signed 64-bit integer.
inline pair<i64, i64> gcd_orbit_shape(i64 n, i64 step) {
  if (n < 1) throw invalid_argument("positive circle size");
  step %= n;
  if (step < 0) step += n;
  i64 cycles = gcd(n, step);
  return {cycles, n / cycles};
}
} // namespace finals_math_extra
```

1.12.4 Base representation becomes divisibility

If N has exactly two equal digits d in base b, then $N = db + d = d(b + 1)$. Enumerate positive divisors d of N, recover $b = N/d - 1$, and require $b \geq 2$ and $1 \leq d < b$. For $N = 12$ this gives $(d, b) = (1, 11), (2, 5), (3, 3)$, but the last pair is illegal because a digit must be smaller than its base. Thus the valid two-digit bases are 11 and 5. This handles exactly two equal digits, not all palindromes. For the broader 2023 F task, smaller bases can have more digits and need direct conversion and a palindrome check; partition the base ranges so results are not duplicated. The proof is the positional numeral equation, not a guess from examples. 2023 official solutions.

1.12.5 Practice and adapt

Counting Divisors: compare a per-query factorization with an SPF or divisor-count sieve at the stated bounds. Counting Coprime Pairs: use E_1, not the number of prime inputs. Test repeated ones and repeated primes. Divisor Analysis: the input already supplies prime exponents; do not reconstruct or factor the huge integer. Its modular divisor-product part requires additional exponent arithmetic. These are practice tasks, not claimed new online-judge acceptances.

1.13 Prime and divisor queries: ready-to-use code

Select by bounds. For many prime queries with maximum endpoint at most 10^7, build `PrimeTable` once. It provides primality, the ordered prime list, and inclusive range counts. Its arrays cost about $5(V + 1)$ bytes plus the prime list, roughly 53 MB at $V = 10^7$ with 4-byte `int`. Check the contest memory limit. For a narrow interval near a large number, use the segmented routine below. It still needs primes through $\sqrt{R}$; a short interval alone does not make arbitrary 64-bit endpoints cheap.

Prelude.h

Description: Shared includes, aliases and bounds. Include before the modules.

`<algorithm>, <cmath>, <stdexcept>, <vector>` 25d466, 6 lines

```
namespace finals_number {
using namespace std;
using ll = long long;

} // namespace finals_number
```

1.13.1 Is x prime? Count or list primes in [L,R]

0 and 1 are not prime. Cross off multiples beginning at p^2: smaller multiples already have a smaller prime factor. Build $P[x] = \#\{p \leq x : p \text{ prime}\}$. Then the answer on inclusive $[L, R]$ is $P[R] - P[L-1]$, with the $L = 0$ boundary handled separately.

PrimeQueries.h

Description: Prime flags, list and inclusive range counts.

`"Prelude.h"` 1b79e5, 38 lines

```
namespace finals_number {
struct PrimeTable {
  vector<char> prime;
  vector<int> prefix, primes;
  // 0<=limit<=10^7. O(V log log V) time, O(V) space.
  explicit PrimeTable(int limit) {
    if (limit < 0 || limit > 10000000)
      throw invalid_argument("prime table limit");
    prime.assign(limit + 1, true);
    prefix.assign(limit + 1, 0);
    prime[0] = false;
    if (limit >= 1) prime[1] = false;
    for (int p = 2; p <= limit / p; ++p)
      if (prime[p])
        for (ll x = 1LL * p * p; x <= limit; x += p)
          prime[x] = false;
    for (int x = 1; x <= limit; ++x) {
      prefix[x] = prefix[x - 1] + prime[x];
      if (prime[x]) primes.push_back(x);
    }
  }
  // Requires 0<=x<=limit; O(1).
  bool is_prime(int x) const { return prime.at(x); }
  // Inclusive [l,r], 0<=l<=r<=limit. O(1).
  int count(int l, int r) const {
    if (l < 0 || r < l || r >= int(prime.size()))
      throw invalid_argument("prime query range");
    return prefix[r] - (l ? prefix[l - 1] : 0);
  }
  // Same bounds; O(log pi(V) + output size).
  vector<int> list(int l, int r) const {
    count(l, r); // check bounds
    auto a = lower_bound(primes.begin(), primes.end(), l);
    auto b = upper_bound(primes.begin(), primes.end(), r);
    return vector<int>(a, b);
  }
};
} // namespace finals_number
```

```
using namespace finals_number;
PrimeTable t(1000000); // build ONCE, outside query loop
bool yes = t.is_prime(97); // true
int count = t.count(10, 30); // 6
vector<int> p = t.list(10, 30);
// p = {11, 13, 17, 19, 23, 29}
```

Build $O(V \log \log V)$; primality and range count $O(1)$; listing takes $O(\log \pi(V) + k)$ for k returned primes. Every queried endpoint must be within the built table. Rebuilding for each query wastes the preprocessing.

Complete query program. Copy Prelude and PrimeQueries above, then add this main. Input: maximum endpoint V, query count Q, then Q pairs L, R. Output: one count per line. All queries obey $0 \le L \le R \le V$.

```
#include <iostream>
int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);
    int V, Q;
    std::cin >> V >> Q;
    finals_number::PrimeTable table(V);
    while (Q--) {
        int L, R;
        std::cin >> L >> R;
        std::cout << table.count(L, R) << '\n';
    }
}
```

For input `30 3` followed by `0 1`, `2 2`, `10 30`, the output lines are 0, 1, 6. A compilable version is `handbooks/examples/prime_range_queries.cpp` in the repository.

1.13.2 Count prime elements in an array interval

This is a different question: count occurrences in array positions, not prime integers in a numeric interval. Duplicates must count repeatedly. After building `t` through the largest nonnegative array value:

```
std::vector<int> pref(a.size() + 1, 0);
for (int i = 0; i < (int)a.size(); ++i)
    pref[i + 1] = pref[i] + (a[i] >= 2 &&
                             t.is_prime(a[i]));
// Array positions [l,r), not numeric values [l,r].
int answer = pref[r] - pref[l];
```

For $a = [2, 4, 2, 7]$, the whole array contains three prime occurrences, although it contains only two distinct prime values. Positions $[1, 4)$ contain two. With point updates, store the prime indicator in a Fenwick tree and update by new-indicator minus old-indicator. The prime table must cover every possible updated value too.

1.13.3 High endpoints: segmented prime enumeration

`segmented_primes(L,R)` marks just the inclusive interval and returns its primes in sorted order. For each base prime $p \le \sqrt{R}$, the first marked multiple is $\max(p^2, \lceil L/p \rceil p)$; using only the rounded multiple would incorrectly mark p itself when it lies in the interval.

SegmentedPrimes.h

Description: Enumerate primes in a high narrow interval.

"Prelude.h", "PrimeQueries.h" 2176e9, 24 lines

```
namespace finals_number {
// Inclusive [L,R]; 0<=L<=R<=10^12; width<=10^7.
// Base sieve to sqrt(R), then mark multiples within the
// window. Time O(B log log B + W log log B + pi(B) + W),
// B=sqrt(R). Space O(B+W), including returned primes.
inline vector<ll> segmented_primes(ll L, ll R) {
  if (L < 0 || R < L || R > 1000000000000LL ||
      R - L + 1 > 10000000)
    throw invalid_argument("segment bounds");
  ll B = sqrt((long double)R);
  while ((B + 1) * (B + 1) <= R) ++B;
  while (B * B > R) --B;
  PrimeTable base{int(B)};
  vector<char> good(size_t(R - L + 1), true);
  for (ll p : base.primes) {
    ll first = max(p * p, ((L + p - 1) / p) * p);
    for (ll x = first; x <= R; x += p) good[x - L] = false;
  }
  vector<ll> answer;
  for (ll x = L; x <= R; ++x)
    if (x >= 2 && good[x - L]) answer.push_back(x);
  return answer;
}
} // namespace finals_number
```

```
auto p = finals_number::segmented_primes(10, 30);
auto count = p.size(); // 6
// Large endpoint, small interval also allowed:
auto high = finals_number::segmented_primes(
    1000000000000LL - 1000, 1000000000000LL);
```

This implementation enforces $0 \le L \le R \le 10^{12}$ and width $W = R - L + 1 \le 10^7$. It needs $O(\sqrt{R} + W)$ space, including output, and scans the base-prime list per call. Many high-interval queries may need shared base preprocessing. To count only and reduce output memory, count surviving flags instead of storing them; the supplied API returns the list. A huge width is not supported just because R is within the endpoint bound. Perfect-square endpoints, 0, 1, and single-number intervals are tested.

1.13.4 How many divisors? What is their sum?

If many positive queries are bounded by V, add each possible divisor d to all multiples of d. This builds the divisor count and divisor sum for every value in $O(V \log V)$ time and $O(V)$ memory. Query in $O(1)$.

DivisorTable.h

Description: Number and sum of divisors for every bounded value.

"Prelude.h" 04fafb, 19 lines

```
namespace finals_number {
struct DivisorTable {
  vector<int> count;
  vector<ll> sum;
  // 0<=limit<=2*10^6. O(V log V) time, O(V) space.
  // Query only positive x<=limit. Slot zero is unused.
  explicit DivisorTable(int limit) {
    if (limit < 0 || limit > 2000000)
      throw invalid_argument("divisor table limit");
    count.assign(limit + 1, 0);
    sum.assign(limit + 1, 0);
    for (int d = 1; d <= limit; ++d)
      for (int x = d; x <= limit; x += d) {
        ++count[x];
        sum[x] += d;
      }
  }
};
} // namespace finals_number
```

```
finals_number::DivisorTable d(1000000);
int count = d.count[12]; // 6
long long sum = d.sum[12]; // 28
long long proper_sum = d.sum[12] - 12; // 16
bool perfect = (d.sum[6] - 6 == 6); // true
// d.count[1] == 1, d.sum[1] == 1
```

Only positive query values are valid; zero has no finite divisor count. At $V = 2 \cdot 10^6$, the two arrays need about 24 MB. If only counts are needed, omit the sums array and its update. For a few large integers, use factorization or divisor enumeration instead of allocating up to their value. The repository also includes runnable `divisor_queries.cpp` and `segmented_primes.cpp` examples beside the prime query program.